Application Area: Customer Service Intelligent System

Dataset: https://www.kaggle.com/datasets/zadafiyabhrami/customer-service-ineraction-dataset

Student Name:

Student Id:

Contents

1. Application and Workflow Selection 4

1.1 Introduction and Motivation of the Selected Application.....4

1.2 Explanation of the Selected Workflow with Diagram.....4

1.3 Objectives of the Selected Intelligent System.....6

1.4 Impact of the Selected Project if Developed.....7

2. Selection of AI Technologies 7

2.1 Identification of Similar Systems 7

2.2 Innovative and Original Approaches 7

2.3 Justification for AI Methods, Technologies, and Tools 7

3. Simulations and Development Setup 8

3.1 Simulation Environments and Setup Parameters 8

3.2 Agent-wise Task Distribution, Functionality, and Datasets 9

3.3 Encoding and Data Processing Prior to Simulation 9

4. Use Case Explanation with Evidence.....9

4.1 Step-by-Step Workflow Execution with Evidence.....9

4.2 Developed User Interface and External Interaction 24

5. Conclusions.....26

References 27

List of Figures

Figure 1 Workflow Diagram.....5
Figure 2 Import Librery.....8
Figure 3 Download data form kaggle.....10
Figure 4 Agent 1 Implementations 1.....11
Figure 5 Agent 1 Implementations 2.....12
Figure 6 Agent 1 Implementations 3.....13
Figure 7 Agent 1 Implementations 3.....14
Figure 8 Agent 1 Output.....15
Figure 9 Bivariate Analysis.....16
Figure 10 Target and Summary Insights.....16
Figure 11 Agent 2 Implementations.....17
Figure 12 Agent 2 Output.....18
Figure 13 Agent 3 Implementations.....19
Figure 14 Agent 3 Output.....20
Figure 15 Agent 4 Implementations.....21
Figure 16 Agent 4 Output.....22
Figure 17 Agent 5 Implementations.....23
Figure 18 Agent 5 Output.....24

1. Application and Workflow Selection

1.1 Introduction and Motivation of the Selected Application

Customer service is now a key point of difference among companies working in a highly competitive digital setting. Customers demand prompt, precise and consistent services to their services inquiry in all the channels. Traditional customer service systems are however extremely manual or strictly governed by specific rules such that the system slows down in processing customer issues that leads to delayed responses, lack of scalability, and lack of consistency in services. With the increment in the volume and complexity of customer interactions, organizations are experiencing great challenges in ensuring high customer satisfaction besides being able to control the cost of their operations.

These challenges form the basis of the reason why customer service was chosen as the area of application in the current project. Customer services are by nature, sequential and multi-stage and can therefore be approached using an Agentic AI. The various processes involved in data intake, preprocessing, reasoning, prediction, evaluation and explanation may be outsourced to independent agents that work together towards a common objective. With the help of an Agentic AI system, customer interactions can be more effectively served, and the corresponding agent is provided with a specific task, which enhances the clarity of the system, modularity, and scalability (Nie et al., 2021).

This project is aimed at the design and implementation of an intelligent customer service system that anticipates the satisfaction levels of customers, regarding the attributes of interactions. The system theorizes how AI agents can assist in decision-making processes of customer service settings through the analysis of past interaction data and the generation of actionable insights.

1.2 Explanation of the Selected Workflow with Diagram

[redacted]

[redacted]

[redacted]

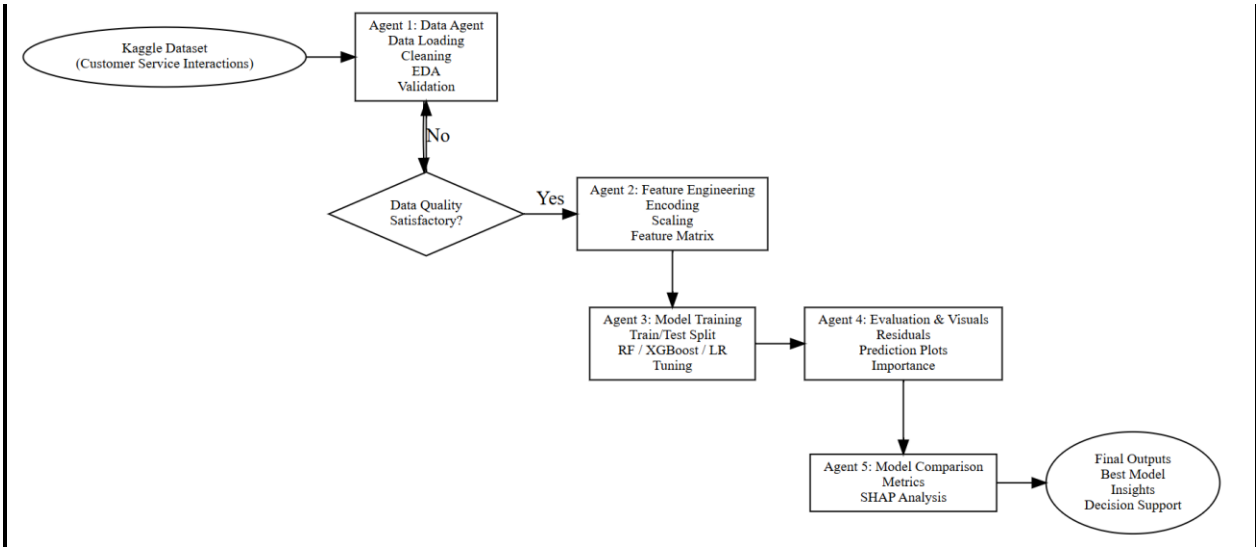
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

Awasthy.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Import Librery

```python
# Imports and reproducibility
import os
import random
import numpy as np
import pandas as pd

# Machine learning
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, precision_recall_curve, confusion_matrix
)
import xgboost as xgb
import joblib
import matplotlib.pyplot as plt
import seaborn as sns

# For class imbalance handling
from imblearn.over_sampling import SMOTE

# For interpretability
import shap

# Reproducibility
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
random.seed(RANDOM_STATE)
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

Awasthy.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

DOWNLOAD DATA FROM KAGGLE

```python
# 1. DOWNLOAD DATA FROM KAGGLE

# Install Kaggle
!pip install -q kaggle

# Create kaggle.json file with credentials
kaggle_token = {
    "username": "your_username",
    "key": "your_kaggle_api_key"
}

import os
import json

# Save credentials
os.makedirs(os.path.expanduser("~/.kaggle"), exist_ok=True)
with open(os.path.expanduser("~/.kaggle/kaggle.json"), "w") as f:
    json.dump(kaggle_token, f)

# Set permission
!chmod 600 /root/.kaggle/kaggle.json

# Download dataset (Customer Service Interaction dataset)
!kaggle datasets download -d zadafiyabhrami/customer-service-ineraction-dataset -p ./data

# Unzip dataset
import zipfile
with zipfile.ZipFile('./data/customer-service-ineraction-dataset.zip', 'r') as z:
    z.extractall('./data/customer_service_interaction')

# Show extracted files
for root, dirs, files in os.walk('./data/customer_service_interaction'):
    print(root, files[:10])
```

```
Dataset URL: https://www.kaggle.com/datasets/zadafiyabhrami/customer-service-ineraction-dataset
License(s): Community Data License Agreement - Sharing - Version 1.0
Downloading customer-service-ineraction-dataset.zip to ./data
  0% 0.00/601k [00:00<?, ?B/s]
100% 601k/601k [00:00<00:00, 999MB/s]
./data/customer_service_interaction ['customer_service_interactions.csv']
```

Agent 1: Data Agent

```python
# Agent 1: Data Agent

import os
from pathlib import Path
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import textwrap
import warnings
warnings.filterwarnings("ignore")
sns.set(style="whitegrid", rc={"figure.dpi": 110})

class DataAgent:
    def __init__(self, df: pd.DataFrame, out_dir="agent1_outputs"):
        self.raw = df.copy()
        self.clean_df = None
        self.out_dir = Path(out_dir)
        self.out_dir.mkdir(parents=True, exist_ok=True)
        self.report = {}


    # Basic info & quality

    def data_info(self):
        info = {
            "shape": self.raw.shape,
            "columns": list(self.raw.columns),
            "dtypes": self.raw.dtypes.astype(str).to_dict(),
            "memory_usage_bytes": int(self.raw.memory_usage(deep=True).sum())
        }
        return info

    def quality_checks(self):
        qc = {}
        qc['missing_count'] = self.raw.isnull().sum().sort_values(ascending=False)
```

[redacted]

```python
        qc['duplicate_rows'] = int(self.raw.duplicated().sum())
        # near-constant columns (unique <=1)
        qc['constant_columns'] = [c for c in self.raw.columns if self.raw[c].nunique(dropna=False) <= 1]
        # near-zero variance (std very small for numeric)
        num = self.raw.select_dtypes(include=[np.number]).columns.tolist()
        nzv = []
        for c in num:
            try:
                if self.raw[c].std(skipna=True) == 0 or (self.raw[c].nunique(dropna=True) / max(1, len(self.raw)) ) < 0.01:
                    nzv.append(c)
            except Exception:
                pass
        qc['near_zero_variance'] = nzv
        return qc


    # Basic cleaning

    def basic_clean(self, strip_cols=True, drop_duplicates=True):
        df = self.raw.copy()
        if strip_cols:
            df.columns = df.columns.str.strip().str.replace(" ", "_").str.replace("-", "_")
        if drop_duplicates:
            before = len(df)
            df = df.drop_duplicates().reset_index(drop=True)
            after = len(df)
            self.report['duplicates_removed'] = before - after
        # try converting obvious numeric-like columns
        for c in df.columns:
            if df[c].dtype == object:
                # attempt numeric conversion if many values look numeric
                sample = df[c].dropna().astype(str).head(200).tolist()
                num_like = sum(1 for v in sample if v.replace(".","",1).replace("-","",1).isdigit())
                if len(sample)>0 and num_like/len(sample) > 0.8:
                    try:
                        df[c] = pd.to_numeric(df[c], errors='coerce')
                        self.report[f'coerced_{c}'] = True
                    except Exception:
                        pass
```

[redacted]

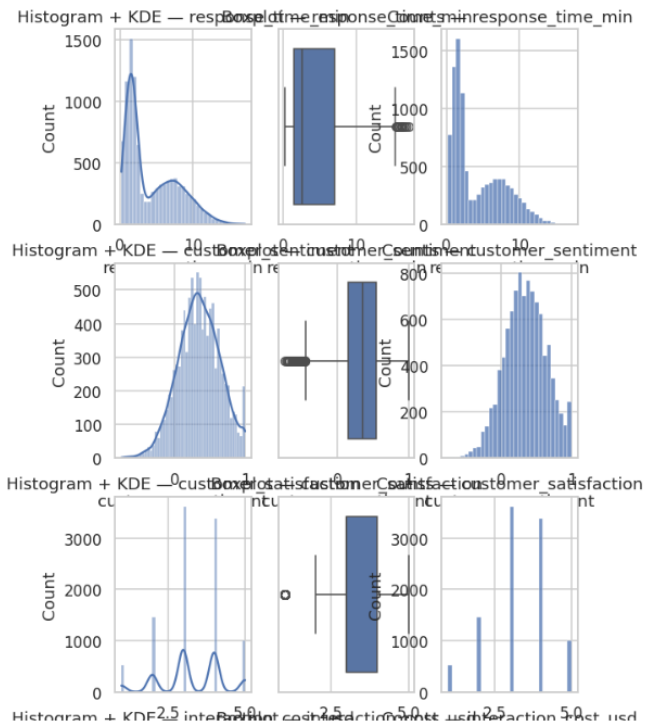
```python
        self.clean_df = df
        return df


    # EDA helpers & plotting

    def _select_columns_for_plot(self, max_num=6):
        """Choose a small representative set of numeric and categorical columns for plotting."""
        num = self.clean_df.select_dtypes(include=[np.number]).columns.tolist()
        cat = self.clean_df.select_dtypes(include=['object','category','bool']).columns.tolist()
        # choose top few by non-null counts
        def top_k(cols, k):
            cols_sorted = sorted(cols, key=lambda c: self.clean_df[c].count(), reverse=True)
            return cols_sorted[:k]
        return top_k(num, min(max_num, len(num))), top_k(cat, min(max_num, len(cat)))

    def eda_univariate(self, save_name="fig_univariate.png", sample_n=10000):
        """
        Figure 1: Univariate analysis combined into subplots
         - Histogram + KDE for numeric features
         - Countplot for top categorical features
         - Boxplots for numeric features
        """
        num_cols, cat_cols = self._select_columns_for_plot(max_num=6)
        n_num = len(num_cols)
        n_cat = len(cat_cols)
        cols = max(1, n_num + n_cat)
        fig = plt.figure(constrained_layout=True, figsize=(6*1, 3*cols))
        gs = fig.add_gridspec(cols, 3)
        i = 0
        # Numeric hist + KDE
        for c in num_cols:
            ax = fig.add_subplot(gs[i, 0])
            data = self.clean_df[c].dropna()
            if len(data) > sample_n:
                data = data.sample(sample_n, random_state=1)
            sns.histplot(data, kde=True, ax=ax)
            ax.set_title(f"Histogram + KDE — {c}")
            ax = fig.add_subplot(gs[i, 1])
            sns.boxplot(x=self.clean_df[c], ax=ax)
```

[redacted]

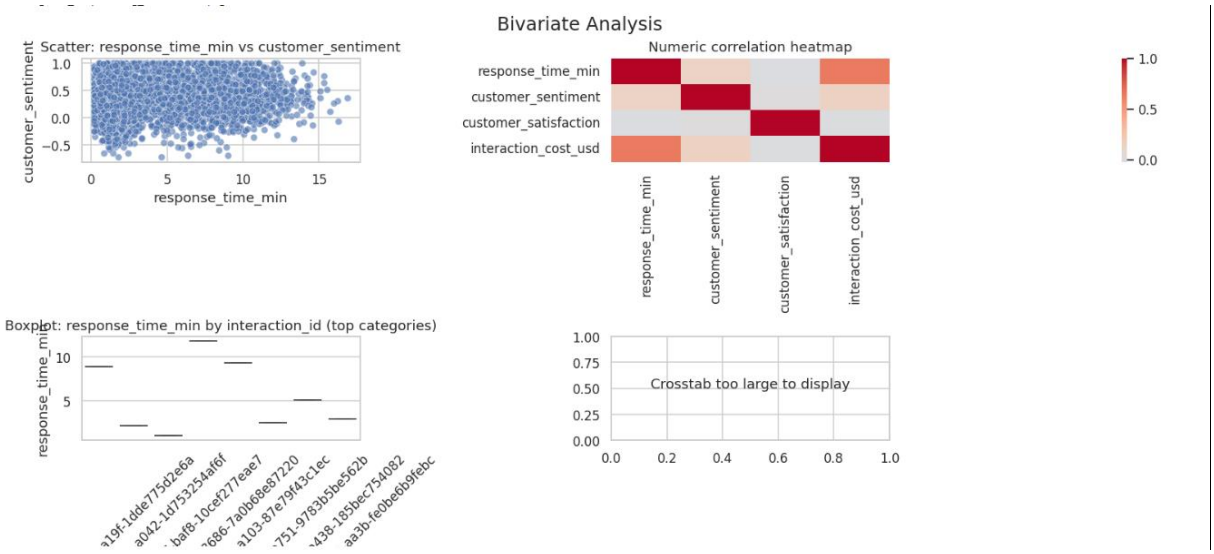
```python
            ax = fig.add_subplot(gs[i, 1])
            sns.boxplot(x=self.clean_df[c], ax=ax)
            ax.set_title(f"Boxplot — {c}")
            ax = fig.add_subplot(gs[i, 2])
            sns.histplot(self.clean_df[c].dropna(), bins=30, ax=ax)
            ax.set_title(f"Counts — {c}")
            i += 1
        # Categorical countplots
        for c in cat_cols:
            ax = fig.add_subplot(gs[i, :])
            topv = self.clean_df[c].value_counts().nlargest(10)
            sns.barplot(x=topv.values, y=topv.index, orient='h', ax=ax)
            ax.set_title(f"Top counts — {c} (top 10)")
            i += 1
        fig.suptitle("Univariate Analysis", fontsize=16)
        fig.savefig(self.out_dir / save_name, bbox_inches='tight')
        plt.show()
        return self.out_dir / save_name

    def eda_bivariate(self, save_name="fig_bivariate.png", sample_n=5000):
        """
        Figure 2: Bivariate analysis combined
         - Scatter plots for selected numeric pairs
         - Boxplots of numeric by top categorical
         - Cross-tab heatmap for two categorical features (if exist)
        """
        num_cols, cat_cols = self._select_columns_for_plot(max_num=6)
        fig = plt.figure(constrained_layout=True, figsize=(14,10))
        gs = fig.add_gridspec(3, 2)
        # Scatter: first two numeric columns
        if len(num_cols) >= 2:
            ax = fig.add_subplot(gs[0,0])
            xcol, ycol = num_cols[0], num_cols[1]
            sub = self.clean_df[[xcol,ycol]].dropna()
            if len(sub) > sample_n:
                sub = sub.sample(sample_n, random_state=1)
            sns.scatterplot(x=xcol, y=ycol, data=sub, alpha=0.6, ax=ax)
            ax.set_title(f"Scatter: {xcol} vs {ycol}")
        else:
            fig.add_subplot(gs[0,0]).text(0.5,0.5,"Not enough numeric cols for scatter", ha='center')
```

[redacted]

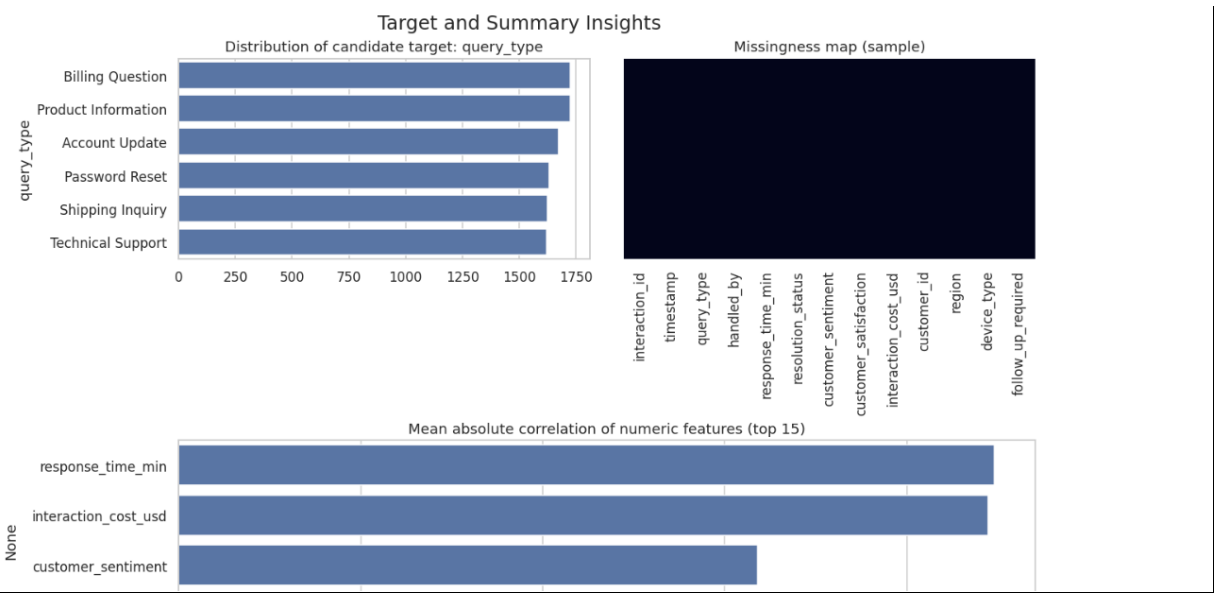
```
>> DataAgent: Starting checks and basic cleaning...
Shape: (10000, 13)
Memory usage (bytes): 5563983
Duplicate rows: 0
Constant columns: []
Top missing values (top 10):
interaction_id           0
timestamp                0
query_type               0
handled_by               0
response_time_min        0
resolution_status        0
customer_sentiment       0
customer_satisfaction    0
interaction_cost_usd     0
customer_id              0

>> Performing basic cleaning...
Clean shape: (10000, 13)
Saved cleaned dataset to: agent1_outputs/cleaned_dataset.csv

>> Generating EDA figures (this can take some seconds)...
```



[redacted]



[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

AGENT 2: FEATURE ENGINEERING

```python
# AGENT 2: FEATURE ENGINEERING


print("Agent 2: Feature Engineering Started")

# Set target variable
target = "customer_satisfaction"

# Separate features and target
y = df[target]
X = df.drop(columns=[target])

print("Target variable selected as customer_satisfaction")
print("Shape of X:", X.shape)
print("Shape of y:", y.shape)

# Identify numeric and categorical columns
numeric_cols = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_cols = X.select_dtypes(include=["object"]).columns.tolist()

print("Numeric Columns:", numeric_cols)
print("Categorical Columns:", categorical_cols)

# Create the preprocessing pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols)
    ]
)

print("Encoding and scaling features")
```

[redacted]

```python
        ("num", StandardScaler(), numeric_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols)
    ]
)

print("Encoding and scaling features")

# Apply transformation
X_transformed = preprocessor.fit_transform(X)

# Preserve feature names
try:
    encoded_cat_names = preprocessor.named_transformers_["cat"].get_feature_names_out(categorical_cols)
except:
    encoded_cat_names = []

final_feature_names = numeric_cols + list(encoded_cat_names)

print("Final feature matrix prepared")
print("Total features after encoding:", len(final_feature_names))

# Convert to DataFrame
import pandas as pd
X_final = pd.DataFrame(X_transformed.toarray() if hasattr(X_transformed, "toarray") else X_transformed,
                       columns=final_feature_names)

print("Shape of final transformed feature set:", X_final.shape)
print("Agent 2: Feature Engineering Completed")
```

```
Agent 2: Feature Engineering Started
Target variable selected as customer_satisfaction
Shape of X: (10000, 12)
Shape of y: (10000,)
Numeric Columns: ['response_time_min', 'customer_sentiment', 'interaction_cost_usd']
Categorical Columns: ['interaction_id', 'timestamp', 'query_type', 'handled_by', 'resolution_status', 'customer_id', 'region',
Encoding and scaling features
Final feature matrix prepared
Total features after encoding: 20247
Shape of final transformed feature set: (10000, 20247)
Agent 2: Feature Engineering Completed
```

[redacted]

[redacted]

[redacted]

AGENT 3: MODEL TRAINING

```python
# AGENT 3: MODEL TRAINING


print("Agent 3: Model Training Started")

from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Step 1: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X_final, y, test_size=0.2, random_state=42)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)

# Step 2: Define models
models = {
    "LinearRegression": LinearRegression(),
    "RandomForest": RandomForestRegressor(random_state=42),
    "XGBoost": XGBRegressor(random_state=42, objective="reg:squarederror")
}

# Step 3: Hyperparameter grids for tuning
param_grids = {
    "RandomForest": {
        "n_estimators": [100, 200, 300],
        "max_depth": [None, 5, 10],
        "min_samples_split": [2, 5, 10]
    },
    "XGBoost": {
        "n_estimators": [100, 200, 300],
        "max_depth": [3, 5, 7],
        "learning_rate": [0.01, 0.05, 0.1],
        "subsample": [0.7, 0.8, 1.0]
    }
}
```

[redacted]

```python
results = {}
for name, model in trained_models.items():
    y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    results[name] = {"MSE": mse, "R2": r2}
    print("Model:", name)
    print("MSE:", mse)
    print("R2 Score:", r2)


print("Agent 3: Model Training Completed")
```

```
Agent 3: Model Training Started
Train shape: (8000, 20247)
Test shape: (2000, 20247)
Training model: LinearRegression
LinearRegression training completed
Training model: RandomForest
Best parameters: {'n_estimators': 200, 'min_samples_split': 5, 'max_depth': 5}
RandomForest training completed
Training model: XGBoost
Best parameters: {'subsample': 0.8, 'n_estimators': 200, 'max_depth': 3, 'learning_rate': 0.05}
XGBoost training completed
Evaluating models on test set
Model: LinearRegression
MSE: 0.7613252166508755
R2 Score: 0.2167564971658127
Model: RandomForest
MSE: 0.7393530286375471
R2 Score: 0.2393612567719594
Model: XGBoost
MSE: 0.7416541576385498
R2 Score: 0.23699384927749634
Agent 3: Model Training Completed
```

[redacted]

[redacted]

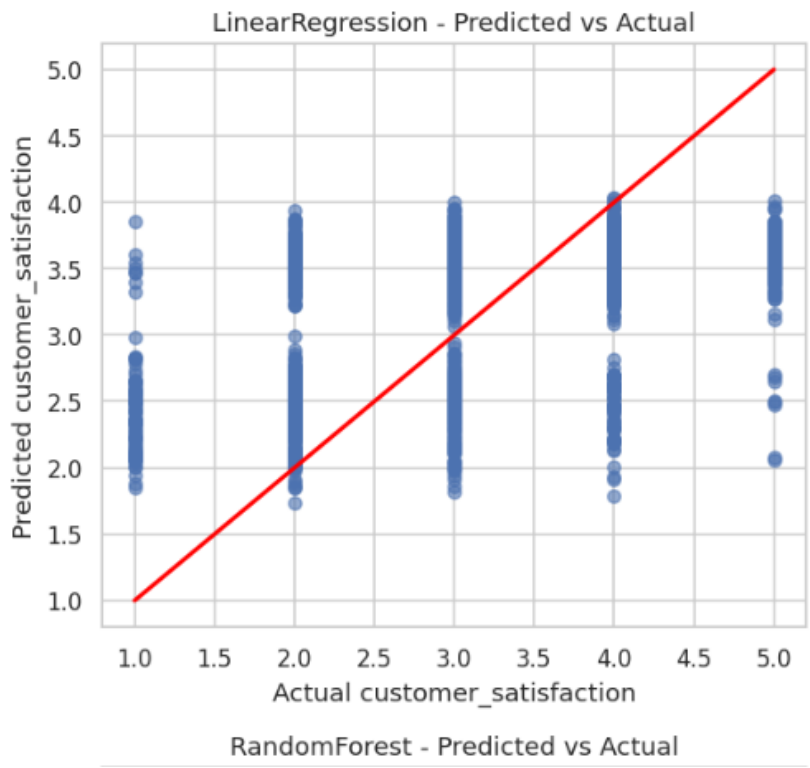
AGENT 4: EVALUATION AND VISUALS

```python
# AGENT 4: EVALUATION AND VISUALS


import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

print("Agent 4: Evaluation and Visuals Started")

# Ensure trained_models exist
try:
    trained_models
except NameError:
    raise Exception("trained_models not found. Run Agent 3 first.")

# Step 1: Predicted vs Actual Plots
for name, model in trained_models.items():
    y_pred = model.predict(X_test)

    plt.figure(figsize=(6, 5))
    plt.scatter(y_test, y_pred, alpha=0.6)
    plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', linewidth=2)
    plt.xlabel("Actual customer_satisfaction")
    plt.ylabel("Predicted customer_satisfaction")
    plt.title(f"{name} - Predicted vs Actual")
    plt.show()

# Step 2: Residual Plots
for name, model in trained_models.items():
    y_pred = model.predict(X_test)
    residuals = y_test - y_pred

    plt.figure(figsize=(6, 5))
    sns.histplot(residuals, kde=True, bins=30)
    plt.xlabel("Residuals")
    plt.title(f"{name} - Residuals Distribution")
    plt.show()

    plt.figure(figsize=(6, 5))
    plt.scatter(y_pred, residuals, alpha=0.6)
```

[redacted]

```python
# Step 4: Summary Table for Model Performance
performance = []
for name, model in trained_models.items():
    y_pred = model.predict(X_test)
    mse = np.mean((y_test - y_pred)**2)
    r2 = 1 - (np.sum((y_test - y_pred)**2)/np.sum((y_test - np.mean(y_test))**2))
    performance.append({"Model": name, "MSE": mse, "R2": r2})

performance_df = pd.DataFrame(performance).sort_values(by="R2", ascending=False)
print("Model Performance Summary")
print(performance_df)

print("Agent 4: Evaluation and Visuals Completed")
```

```
Agent 4: Evaluation and Visuals Started
```



[redacted]

[redacted]

[redacted]

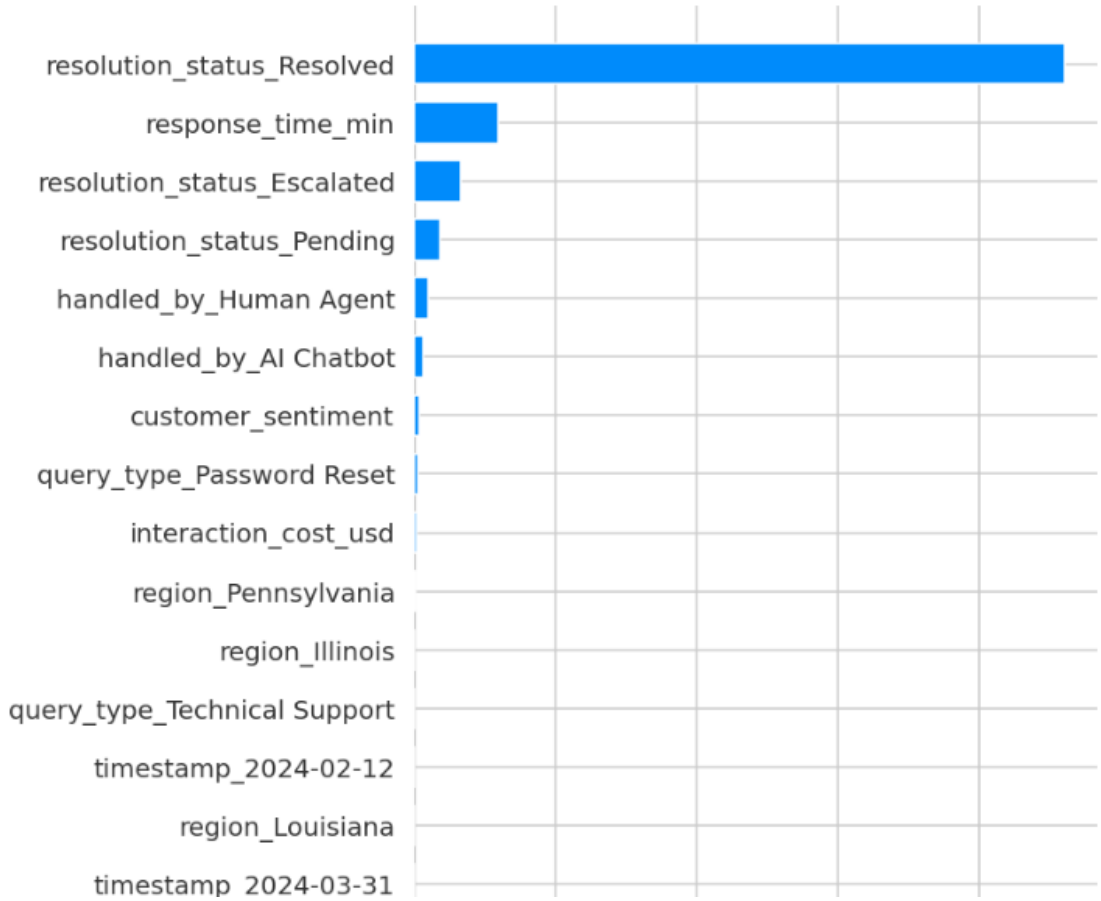
[redacted]

Agent 5 — Model Comparison and SHAP Feature Analysis

```python
# ============================
# AGENT 5: MODEL COMPARISON AND FEATURE SELECTION
# ============================

import shap
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

print("Agent 5: Model Comparison and Feature Selection Started")

# Ensure trained_models exist
try:
    trained_models
except NameError:
    raise Exception("trained_models not found. Run Agent 3 first.")

# Step 1: Calculate performance metrics
performance = []
for name, model in trained_models.items():
    y_pred = model.predict(X_test)
    mse = np.mean((y_test - y_pred)**2)
    r2 = 1 - (np.sum((y_test - y_pred)**2) / np.sum((y_test - np.mean(y_test))**2))
    performance.append({"Model": name, "MSE": mse, "R2": r2})

performance_df = pd.DataFrame(performance).sort_values(by="R2", ascending=False)
print("Model Performance Summary")
print(performance_df)

# Step 2: Select best model
best_model_name = performance_df.iloc[0]["Model"]
best_model = trained_models[best_model_name]

print(f"Best model selected based on R2: {best_model_name}")

# Step 3: SHAP Analysis (for tree-based models)
if hasattr(best_model, "feature_importances_"):
```

[redacted]

```
Agent 5: Model Comparison and Feature Selection Started
Model Performance Summary
              Model       MSE        R2
1      RandomForest  0.739353  0.239361
2           XGBoost  0.741654  0.236994
0  LinearRegression  0.761325  0.216756
Best model selected based on R2: RandomForest
Computing SHAP values for best model
 98%|==================| 1965/2000 [00:54<00:00]
```



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]